

Headway App Architecture Research

https://gemini.google.com/app/525d63d1cb319c8e

User prompt: Do a research on app called headway. design, functions, architecture and everything so that i can build a copy of it

Response: I've put together a research plan. If you need to update it in any way, let me know.

User prompt: Start research

Response: I've completed your research. Feel free to ask me follow-up questions or request changes.

User prompt: i wanna make a copy of this app but it is just for myself. it could be just PWA. but i want it to be as good as the original. the cheaper the better. can you design it please? ask me anything you need to design the app.

Response: Here is the complete architectural and UX design for building a **personal, zero-to-low-cost Progressive Web App (PWA)** clone of Headway that matches the native feel and core retention features of the original app.

1. High-Level Technical Architecture (\$0 Operating Cost)

Because this is strictly for your personal use, you can eliminate the entire multi-tenant infrastructure, payroll layers, and payment systems, replacing them with a **local-first, static-first architecture** hosted entirely on free tiers.

Layer	Recommended Technology	Why It Fits Personal Use & Low Cost	Monthly Cost
Frontend Framework	React / Vite or Next.js (Static Export) + Tailwind CSS	Fast bundle size, instant hot-reloading, simple PWA packaging via vite-plugin-pwa or @ducanh2912/next-pwa.	\$0
Hosting & CDN	Cloudflare Pages or Vercel	Free global CDN edge hosting with unlimited bandwidth for personal traffic and automated Git deployments.	\$0
Storage & Database	IndexedDB (via Dexie.js) for local storage; optional Supabase free tier if syncing across multiple devices	Pure local-first data means zero server bills, instant loading, complete privacy, and full offline functionality.	\$0
Audio Generation	edge-tts (Python/CLI) or Web Speech API	edge-tts provides Microsoft’s natural neural voices for free without API limits; Web Speech API provides zero-setup on-device playback.	\$0
Content Generation	Local LLM (Ollama: Llama 3) or Gemini Flash / Claude 3.5 Haiku API	High-speed, high-context summarization scripts costing fractions of a cent per book.	\$0 to <\$0.50/month

2. Core Functional Modules & UI Components

A. Mobile PWA Shell & Design System

- **Display Mode:** Configured in `manifest.json` as `"display": "standalone"` with `"orientation": "portrait"`, hiding browser chrome so it looks like a native iOS/Android application.
- **Color Palette:**
 - Background: #0F1115 (Deep Charcoal Dark Mode) or #F8F9FA (Clean Light Mode).
 - Brand Accent: #FFDE30 (Headway Signature Yellow) for CTAs, active streak indicators, and player progress.
 - Cards & Surfaces: #1A1D24 (Elevated dark card).
- **Typography:** Clean humanist sans-serif (Inter or system fonts `-apple-system`, `BlinkMacSystemFont`) with a line height of $1.5\times$ for summary readability.

B. Synchronized Audio & Text Reader Engine

- **Sentence-Level Highlighting:** The summary text is split into paragraphs and sentences tagged with millisecond offsets (`data-start` and `data-end`). As HTML5 audio plays, a `timeupdate` listener applies a highlight style to the active sentence in real time.
- **Native Lockscreen Integration (Media Session API):**
 - Implements `navigator.mediaSession.metadata` so lock screens, dynamic islands, Apple Watch, and Bluetooth headsets show the book title, custom cover artwork, chapter name, and scrub controls.
 - Configures action handlers: `play`, `pause`, `seekforward (15s)`, `seekbackward (15s)`, and `previous track / next track`.
- **Audio Controls:** Variable playback rate ($0.75\times$ to $2.0\times$) and an adjustable sleep timer.

C. Active Recall & Spaced Repetition (SM-2 Algorithm)

- **Flashcard Engine:** When reading, selecting text opens an instant "Add Highlight" or "Create Flashcard" bottom sheet.
- **Algorithmic Review:** Each card stores:
 - Repetition count (n)
 - Ease factor (EF , initialized at 2.5)
 - Interval in days (I)
 - Next review timestamp ($Date$)
- Users rate recall as **Hard (2)**, **Good (4)**, or **Easy (5)**, updating intervals locally via the classic SuperMemo-2 formula.

D. Habit Gamification & "Shorts" Feed

- **Streak Counter:** Tracks consecutive calendar days with at least 3 minutes of reading/listening logged in IndexedDB.
- **Interactive Shorts:** A vertical swipeable card view (using Framer Motion or CSS scroll snap) pulling actionable takeaways, one-sentence mental models, and scenario quizzes.

3. Automated Content & Audio Pipeline

Instead of writing and recording summaries manually, run a simple two-step offline generation script to populate your app:

Step 1: LLM Summarization Script (Python / Node.js)

Feed your EPUB, PDF, or book title into a script that runs a prompt enforcing Headway's structural format:

- **Format:** 1 Introduction, 5 core conceptual chapters (~350 words each), and 1 actionable conclusion.
- **Extraction:** 5 standalone "Insight Gems" (formatted as front/back Q&A flashcards) and 1 practical scenario question for the Shorts feed.
- **Output:** Structured JSON document.

Step 2: Zero-Cost Neural Voice Narration

Use edge-tts (which connects to Microsoft's neural voice engine without requiring API keys):

- Synthesizes your summary script into an .mp3 file using natural voices like en-US-GuyNeural or en-US-JennyNeural.
- Generates synchronized sentence-level timecodes using word/sentence boundaries directly into a .json or .vtt file for synchronized reader highlighting.

4. Database Schema (IndexedDB / Local-First)

Using Dexie.js, your client-side database schema maps directly to:

- **books:** id, title, author, category, total_duration_sec, cover_data_url
- **chapters:** id, book_id, chapter_index, title, content_text, audio_timestamp_start, audio_timestamp_end
- **user_progress:** book_id, last_chapter_id, audio_position_ms, is_completed, updated_at
- **flashcards:** id, book_id, front_text, back_text, ease_factor, interval_days, repetitions, next_review_at
- **streaks:** date, minutes_consumed

Questions to Refine the Implementation

To tailor the codebase, stack, and setup instructions to your exact technical background, please share:

1. Content Ingestion: How do you plan to supply books?

- *Option A:* Upload your own EPUB/PDF files and let the app summarize them.
- *Option B:* Type a book title/author and have an AI script automatically generate the summary from its training knowledge.
- *Option C:* Manually write or copy-paste text summaries you already have.

2. Device Usage & Syncing:

- Will you use this **exclusively on one phone** (pure local IndexedDB, 100% offline, zero setup)?
- Or do you need **cross-device sync** between your phone, tablet, and desktop (which is easily handled by setting up a free-tier Supabase backend)?

3. Preferred Tech Stack:

- Do you have a preferred frontend library (e.g., React, Next.js, Vue, Svelte, or vanilla HTML/JS)?

4. Audio Narration Preference:

- Do you prefer pre-generating high-quality audio files using an offline script (like edge-tts)?
- Or would you prefer on-the-fly streaming speech directly inside the browser using the browser's built-in Web Speech synthesis?

User prompt: can you extract transcript from youtubes like a diary of a CEO as source of content? and AI makes summary of it. i will need it on one phone. but would be good if the contetn is saved in cloud i have no preference on tech stack. just easy for AI agents. AUdio narration would be nice

Response: Here is the complete system design and implementation blueprint for your personal, zero-cost Headway clone.

This design uses **React + Vite + Tailwind** for the PWA (the most reliable stack for AI coding agents like Cursor or Claude), **Supabase** (free tier) for cloud database and audio storage, and a **local Python ingestion script** that extracts YouTube transcripts from podcasts like *The Diary of a CEO*, summarizes them into Headway's 5-chapter format, generates natural neural audio for free, and pushes everything to the cloud.

System Architecture Overview

- **Ingestion Runner (Local Python Script / Agent):** Takes a YouTube URL, pulls the transcript via your home IP (bypassing YouTube's cloud datacenter blocking), sends it to an LLM (such as Gemini Flash) to generate the structured summary and flashcards, uses edge-tts to generate studio-grade narration for free, and uploads the audio and JSON to Supabase.
- **Cloud Backend (Supabase Free Tier):**
 - **Database (PostgreSQL):** Stores summaries, chapters, flashcards, and reading streaks.
 - **Storage Buckets:** Hosts generated .mp3 audio files and cover art.
- **Frontend Mobile PWA (React + Vite + Tailwind):**
 - Deployed to **Cloudflare Pages** or **Vercel** (\$0/month).
 - Installed to your phone's home screen as a standalone app.
 - Features the synchronized audio/text player, lockscreen media controls, spaced repetition (SM-2) flashcard deck, and habit streak tracker.

Step 1: Content Ingestion Pipeline (YouTube to Headway Format)

Because YouTube blocks automated requests originating from cloud IP ranges (AWS, Google Cloud, Vercel), running this extraction script on your laptop or local environment ensures 100% success.

Ingestion Workflow

1. **Fetch Transcript:** Uses youtube-transcript-api to pull full subtitles with timestamps.
2. **AI Summarization:** A single prompt structures a 2-hour podcast (25,000+ words) into:

- 1 Introduction
- 5 Core conceptual chapters (~350 words each)
- 1 Actionable recap
- 5 "Insight Gems" (formatted as spaced repetition Q&A flashcards)
- 1 Interactive scenario quiz for the Shorts feed

3. **Audio Synthesis (edge-tts):** Generates an .mp3 file using natural voices like en-GB-RyanNeural (fits *The Diary of a CEO* style) or en-US-GuyNeural without API costs.

4. **Cloud Upload:** Pushes the audio to a Supabase Storage bucket and writes records to PostgreSQL.

Ingestion Script (pipeline.py)

```
import os
import json
import asyncio
from youtube_transcript_api import YouTubeTranscriptApi
import edge_tts
from google import genai
from supabase import create_client

# Credentials
SUPABASE_URL = "https://your-project.supabase.co"
SUPABASE_KEY = "your-service-role-key"
GEMINI_API_KEY = "your-gemini-key"

supabase = create_client(SUPABASE_URL, SUPABASE_KEY)
ai_client = genai.Client(api_key=GEMINI_API_KEY)

def get_transcript(video_id: str) -> str:
    transcript_list = YouTubeTranscriptApi.get_transcript(video_id)
    return "".join([item["text"] for item in transcript_list])

def summarize_to_headway_format(raw_transcript: str) -> dict:
    prompt = f"""
    You are an expert editorial writer for the Headway microlearning app.
    Analyze this podcast transcript and produce a structured microlearning summary.

    Format requirements:
    1. title: Title of the summary
    2. author_or_guest: Name of the guest/speaker
    3. duration_min: Estimated read time (usually 15)
    4. chapters: Array of exactly 6 items (1 Intro, 4 thematic concept chapters, 1 Actionable Recap).
       Each chapter must have:
       - "chapter_index": Integer
       - "title": Compelling chapter title
       - "content": 300 to 400 words of punchy, practical prose.
    5. flashcards: Array of 5 key insight flashcards:
       - "front": Core question or concept
       - "back": The punchy takeaway or behavioral rule
    6. quiz: One scenario-based quiz:
       - "scenario": Real-world dilemma
       - "options": Array of 3 choices
       - "correct_index": Index (0-2)
       - "explanation": Brief lesson explaining why

    Output strictly valid JSON matching these keys.

    TRANSCRIPT:
    {raw_transcript[:60000]}
    """
    response = ai_client.models.generate_content(
        model="gemini-2.0-flash",
        contents=prompt,
        config={"response_mime_type": "application/json"}
    )
    return json.loads(response.text)

async def generate_narration(chapters: list, output_filename: str):
```

```

full_text = " ".join([f"{ch['title']}. {ch['content']}" for ch in chapters])
communicate = edge_tts.Communicate(full_text, voice="en-GB-RyanNeural")
await communicate.save(output_filename)

async def process_video(video_id: str):
    print("1. Extracting YouTube transcript...")
    transcript = get_transcript(video_id)

    print("2. Generating Headway-style synthesis...")
    summary_data = summarize_to_headway_format(transcript)

    print("3. Generating neural voice narration...")
    audio_path = f"{video_id}.mp3"
    await generate_narration(summary_data["chapters"], audio_path)

    print("4. Uploading audio to Supabase Storage...")
    with open(audio_path, "rb") as f:
        supabase.storage.from_("audio").upload(f"{video_id}.mp3", f)
    audio_public_url = supabase.storage.from_("audio").get_public_url(f"{video_id}.mp3")

    print("5. Saving records to PostgreSQL...")
    book_res = supabase.table("books").insert({
        "youtube_id": video_id,
        "title": summary_data["title"],
        "author": summary_data["author_or_guest"],
        "audio_url": audio_public_url,
        "total_duration_sec": 900
    }).execute()
    book_id = book_res.data[0]["id"]

    for ch in summary_data["chapters"]:
        supabase.table("chapters").insert({
            "book_id": book_id,
            "chapter_order": ch["chapter_index"],
            "title": ch["title"],
            "content": ch["content"]
        }).execute()

    for fc in summary_data["flashcards"]:
        supabase.table("flashcards").insert({
            "book_id": book_id,
            "front": fc["front"],
            "back": fc["back"]
        }).execute()

    print(f"Done! Title '{summary_data['title']}' is ready in your PWA.")

if __name__ == "__main__":
    asyncio.run(process_video("PASTE_YOUTUBE_VIDEO_ID_HERE"))

```

Step 2: Supabase Cloud Database Schema

Run this SQL script in your Supabase SQL Editor:

```

-- Books / Summaries
create table books (
  id uuid primary key default gen_random_uuid(),
  youtube_id text unique,
  title text not null,
  author text not null,
  audio_url text,
  total_duration_sec int default 900,
  created_at timestamp with time zone default timezone('utc'::text, now())
);

-- Chapters
create table chapters (
  id uuid primary key default gen_random_uuid(),
  book_id uuid references books(id) on delete cascade,
  chapter_order int not null,
  title text not null,
  content text not null
);

-- Spaced Repetition Flashcards (SuperMemo-2)
create table flashcards (
  id uuid primary key default gen_random_uuid(),

```

```

    book_id uuid references books(id) on delete cascade,
    front text not null,
    back text not null,
    ease_factor numeric default 2.5,
    interval_days int default 1,
    repetitions int default 0,
    next_review_at date default current_date
);

-- Daily Streaks & Habits
create table user_activity (
  activity_date date primary key default current_date,
  minutes_read int default 0,
  completed boolean default false
);

```

Step 3: Frontend PWA Architecture (React + Vite)

The frontend is a lightweight Single Page Application configured as a PWA.

Application Layout Structure

- **Bottom Navigation Bar:**
 - **Today (Home):** Daily goal progress, current streak counter, and the active reading queue.
 - **Reader / Player:** The core reading screen with embedded audio controls, font resizing, and lockscreen integration.
 - **Spaced Repetition:** Flashcard review deck using the SM-2 algorithm.
 - **Library:** All synthesized podcast summaries categorized by topic.

PWA Configuration (vite.config.ts)

Using vite-plugin-pwa, the application acts like an installed native app on iOS/Android:

```

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate',
      manifest: {
        name: 'Headway Clone',
        short_name: 'Headway',
        start_url: '/',
        display: 'standalone',
        background_color: '#0F1115',
        theme_color: '#0F1115',
        icons: [
          { src: '/icon-192.png', sizes: '192x192', type: 'image/png' },
          { src: '/icon-512.png', sizes: '512x512', type: 'image/png' }
        ]
      }
    })
  ]
});

```

Synchronized Audio Engine with Lockscreen Controls

To give your web app the native audio feel of Headway, integrate the browser's `MediaSession` API:

```

// AudioPlayer.jsx
import React, { useRef, useEffect } from 'react';

```

```

export function AudioPlayer({ currentBook, activeChapter }) {
  const audioRef = useRef(null);

  useEffect(() => {
    if (!('mediaSession' in navigator) || !currentBook) return;

    // Configures native iOS/Android lockscreen and Control Center widgets
    navigator.mediaSession.metadata = new MediaMetadata({
      title: activeChapter?.title || currentBook.title,
      artist: currentBook.author,
      album: currentBook.title,
      artwork: [{ src: '/cover-placeholder.png', sizes: '512x512', type: 'image/png' }]
    });

    navigator.mediaSession.setActionHandler('play', () => audioRef.current.play());
    navigator.mediaSession.setActionHandler('pause', () => audioRef.current.pause());
    navigator.mediaSession.setActionHandler('seekforward', () => {
      audioRef.current.currentTime += 15;
    });
    navigator.mediaSession.setActionHandler('seekbackward', () => {
      audioRef.current.currentTime -= 15;
    });
  }, [currentBook, activeChapter]);

  return (
    <div className="fixed bottom-16 left-0 right-0 bg-[#1A1D24] border-t border-gray-800 p-4">
      <audio ref={audioRef} src={currentBook?.audio_url} controls className="w-full h-8" />
    </div>
  );
}

```

Spaced Repetition (SM-2 Algorithm Logic)

When reviewing insight gems, update review intervals locally and sync them back to Supabase:

```

// sm2.ts
export function calculateSM2(quality: number, repetitions: number, easeFactor: number, interval: number) {
  // quality: 2 = Hard, 4 = Good, 5 = Easy
  let nextInterval: number;
  let nextRepetitions: number;
  let nextEaseFactor: number;

  if (quality >= 3) {
    if (repetitions === 0) nextInterval = 1;
    else if (repetitions === 1) nextInterval = 6;
    else nextInterval = Math.round(interval * easeFactor);
    nextRepetitions = repetitions + 1;
  } else {
    nextInterval = 1;
    nextRepetitions = 0;
  }

  nextEaseFactor = easeFactor + (0.1 - (5 - quality) * (0.08 + (5 - quality) * 0.02));
  if (nextEaseFactor < 1.3) nextEaseFactor = 1.3;

  const nextReviewDate = new Date();
  nextReviewDate.setDate(nextReviewDate.getDate() + nextInterval);

  return {
    nextReviewDate: nextReviewDate.toISOString().split('T')[0],
    interval: nextInterval,
    repetitions: nextRepetitions,
    easeFactor: Number(nextEaseFactor.toFixed(2))
  };
}

```

Step 4: Ready-to-Use Prompt for Your AI Agent

You can copy and paste this prompt directly into **Cursor, Claude 3.7 / Sonnet, or ChatGPT** to scaffold the entire frontend in minutes:

Build a mobile-first Progressive Web App (PWA) clone of the "Headway" book summary app using React, Vite, and Tailwind CSS. Connect it to Supabase for data storage.

Key UI and Design Requirements:

1. Theme: Dark theme (#0F1115 canvas, #1A1D24 card surface, #FFDE30 Headway yellow accent, #FAFAFA text).
2. Navigation: Fixed bottom bar with 4 tabs: "Today", "Library", "Review (Cards)", and "Profile".
3. Screens:
 - "Today Screen": Displays a daily habit streak flame counter, a 15-minute daily progress bar, and a "Listen Now" hero card for the latest summary.
 - "Library Screen": Displays cards for books/summaries fetched from Supabase.
 - "Reader/Player Screen": Clean reader view with chapter tabs across the top. An embedded audio player controls playback with a 15-second skip forward/back button and integrates with the browser navigator.mediaSession API for lockscreen metadata.
 - "Flashcard Review Screen": Card flip animation for insight gems with three response buttons: "Hard", "Good", and "Easy", which calculate next review intervals via the SM-2 algorithm.
4. Database Client: Use @supabase/supabase-js to fetch from tables: 'books', 'chapters', 'flashcards', and 'user_activity'.

Operational Cost Breakdown

- **Hosting (Cloudflare Pages / Vercel):** \$0 (Free plan).
 - **Database & Audio Storage (Supabase):** \$0 (Free plan up to 500MB DB / 1GB file storage).
 - **Voice Generation (edge-tts):** \$0 (Unlimited free neural narration).
 - **YouTube Transcript API:** \$0 (Free open-source library running on local IP).
 - **AI Summarization (Gemini 2.0 Flash):** \$0 (Free tier provides 15 RPM, easily handling several podcast summaries per day).
-